

Unsupervised Model Selection for ER

Silhouette, Penalized Silhouette, Modularity, Stability, and Friends

ERBot Toolkit

October 3, 2025

Context: Why Internal Criteria?

- Entity Resolution (ER) often lacks ground truth $\Rightarrow$ need **internal** criteria.
- Compare clusterings across hyperparameters: k (k-means), k_{NN} (graph), thresholds, etc.
- Watch out for **high-dimensional text** behavior: many criteria favor fragmentation or very large k .

Silhouette

Silhouette Score

Given a point i :

$a(i)$ = avg. distance to points in its cluster, $b(i) = \min_{C \neq C_i}$ avg. distance from i to C ,

$$s(i) = \frac{b(i) - a(i)}{\max\{a(i), b(i)\}} \in [-1, 1], \quad S = \frac{1}{n} \sum_i s(i).$$

Heuristic

Higher S indicates tight, well-separated clusters.

Caveat in text ER

As k increases, $a(i)$ shrinks faster than $b(i)$; singletons yield $s(i) \approx 1$. S can **monotonically increase**.

Penalized Silhouette

Penalized Silhouette (ER-friendly)

Idea: Reduce the bias toward fragmentation by penalizing tiny clusters and global over-splitting.

$$S_{\text{pen}} = \underbrace{\text{Agg}(s(i))}_{\text{mean/median silhouette}} - \lambda \cdot \underbrace{\text{Prop}\{|C| < m\}}_{\text{fragmentation penalty}},$$

- Point-wise cap: for singletons or small clusters, set $s(i) \leftarrow \min(s(i), 0)$.
- Global penalty: $\lambda \in [0, 1]$; m is minimum cluster size (e.g., $m = 2$).

Effect

Creates a **peak** vs. k ; discourages trivial splits and singletons dominating S .

CH / DB / Gap

Calinski–Harabasz (CH) & Davies–Bouldin (DB)

Calinski–Harabasz (maximize):

$$\text{CH} = \frac{\text{tr}(B)/(K - 1)}{\text{tr}(W)/(n - K)},$$

where B = between-cluster scatter, W = within-cluster scatter.

Davies–Bouldin (minimize):

$$\text{DB} = \frac{1}{K} \sum_{k=1}^K \max_{\ell \neq k} \frac{S_k + S_\ell}{M_{k\ell}},$$

S_k = intra-cluster scatter, $M_{k\ell}$ = inter-cluster separation.

Notes

Require comparable feature scales; can be unstable in very high dimension without normalization.

$$\text{Gap}(k) = \mathbb{E}_{\text{null}} [\log W_k] - \log W_k,$$

where W_k is within-cluster dispersion, null is a reference (e.g., uniform in PCA box).

- Choose k by the “Tibs2001SEmax” rule: pick smallest k s.t.
 $\text{Gap}(k) \geq \text{Gap}(k+1) - s \cdot SE(k+1)$.
- Computationally heavier (bootstrap over B reference sets).

Graph Modularity

Graph Modularity (Louvain/Leiden on kNN Graph)

Build a kNN graph on embeddings Z (cosine or Euclidean), then run community detection:

$$Q = \frac{1}{2m} \sum_{ij} \left(A_{ij} - \frac{k_i k_j}{2m} \right) \mathbf{1}[g_i = g_j],$$

maximize Q over partitions $\{g\}$.

- kNN parameters: k_{NN} , mutual vs. union, weighted edges.
- Choose k_{NN} by **max modularity**.

Caveat

Resolution limit: very small communities may be merged; tune k_{NN} and use Leiden for better refinement.

Resample Stability

Resample Stability (ARI-based)

Procedure for each hyperparameter setting (e.g., k or k_{NN}):

1. Fit clustering on full data $\Rightarrow$ labels $L^{(0)}$.
2. For $b = 1..B$: subsample fraction ρ , refit $\Rightarrow L^{(b)}$.
3. Compute ARI between $L^{(0)}$ and $L^{(b)}$ on overlapping indices; average.

$$\text{Stability}(k) = \frac{1}{B} \sum_{b=1}^B \text{ARI} \left(L_{\mathcal{I}_b}^{(0)}, L^{(b)} \right).$$

Use

Pick hyperparameters with **max average ARI**. Reflects robustness to sampling noise.

Putting It Together

Comparison: What to Optimize When?

- **Text ER (TF-IDF/SVD, embeddings):** prefer **Penalized Silhouette, Stability, Modularity**.
- **Vector data with good scaling:** CH (max), DB (min), Gap (maxSE).
- **Graphs (kNN + Louvain/Leiden):** Modularity (max) and/or Stability (max).
- Blend scores: e.g., $S_{\text{blend}} = Q + \alpha \cdot \text{Stability}$.

R Snippets

R: Penalized Silhouette (point + global penalties)

```
penalized_silhouette <- function(X, labels,
  dist_method=c("euclidean","cosine"),
  min_cluster_size=2, agg=c("mean","median"),
  lambda=0.3, penalty=c("singleton","size","none"),
  sample_n=NULL, seed=1) {

  dist_method <- match.arg(dist_method); agg <- match.arg(agg)
  penalty <- match.arg(penalty); stopifnot(nrow(X)==length(labels))
  if (!is.null(sample_n) && sample_n < nrow(X)) {
    set.seed(seed); idx <- sample.int(nrow(X), sample_n)
    X <- X[idx,,drop=FALSE]; labels <- labels[idx]
  }
  D <- if (dist_method=="euclidean") dist(X) else
    dist(X, method=function(a,b) 1 - sum(a*b)/sqrt(sum(a^2)*sum(b^2)))
  lab <- as.integer(as.factor(labels))
  s <- cluster::silhouette(lab, D)[,"sil_width"]
  sizes <- table(lab)
  if (penalty=="singleton") s[sizes[as.character(lab)]==1] <- pmin(s[sizes[as.character(lab)]==1], 0)
  if (penalty=="size") s[sizes[as.character(lab)]<min_cluster_size] <- pmin(s[sizes[as.character(lab)]<min_cluster_size], 0)
```

R: CH/DB/Gap Across k (k-means)

```
internal_indices_ch_db <- function(X, labels) {  
  part <- as.integer(as.factor(labels))  
  crit <- clusterCrit::intCriteria(as.matrix(X), part,  
                                   c("Calinski_Harabasz", "Davies_Bouldin"))  
  list(ch=unname(crit$calinski_harabasz), db=unname(crit$davies_bouldin))  
}  
  
compute_gap <- function(X, k_grid, nstart=10, B=20, seed=1) {  
  set.seed(seed)  
  g <- cluster::clusGap(as.matrix(X),  
                        FUNcluster=function(x,k) kmeans(x, k, nstart=nstart),  
                        K.max=max(k_grid), B=B)  
  data.frame(k=g$Tab[, "k"], gap=g$Tab[, "gap"], se_gap=g$Tab[, "SE.sim"])  
}
```

R: Modularity & Stability (Graph + kNN)

```
row_normalize <- function(X){ nr <- sqrt(rowSums(X^2)); nr[nr==0] <- 1; X/nr }
build_knn_graph <- function(Z,k=50,metric=c("cosine","euclidean"),
  mutual=TRUE,weights=TRUE,seed=1){
  set.seed(seed); metric <- match.arg(metric)
  X <- if(metric=="cosine") row_normalize(as.matrix(Z)) else as.matrix(Z)
  nn <- FNN::get.knn(X,k=k)
  edges <- data.frame(from=rep(1:nrow(X),each=k),
    to=as.vector(nn$nn.index),
    dist=as.vector(nn$nn.dist))
  edges <- edges[edges$from!=edges$to,]
  if(mutual){
    key <- paste(edges$from,edges$to,sep="_")
    keep <- key %in% paste(edges$to,edges$from,sep="_"); edges <- edges[keep,]
  }
  if(weights){
    sim <- if(metric=="cosine") pmax(0,1 - (edges$dist^2)/2) else
      exp(-edges$dist/(median(edges$dist)+1e-8))
  } else sim <- NULL
  g <- igraph::graph_from_data_frame(data.frame(from=edges$from,to=edges$to,weight=sim),
    directed=FALSE)
```

R: Resample Stability (Average ARI)

```
stability_via_resamples <- function(X, get_fit, B=20, sample_frac=0.8, seed=1, ...){
  set.seed(seed); n <- nrow(X); ref <- get_fit(X, ...)
  vals <- numeric(B)
  for(b in 1:B){
    idx <- sort(sample.int(n, max(2L, floor(sample_frac*n))))
    vals[b] <- mclust::adjustedRandIndex(ref[idx], get_fit(X[idx,,drop=FALSE], ...))
  }
  mean(vals, na.rm=TRUE)
}

# K-means example:
get_fit_kmeans <- function(X,k,nstart=10){ kmeans(X, k, nstart=nstart)$cluster }

# Graph + Louvain example:
get_fit_graph <- function(X,kNN,metric="cosine",mutual=TRUE,weights=TRUE,seed=1){
  g <- build_knn_graph(X,k=kNN,metric=metric,mutual=mutual,weights=weights,seed=seed)
  igraph::membership(igraph::cluster_louvain(g, weights=igraph::E(g)$weight))
}
```

Usage Playbook

Minimal Usage Examples - 1

```
# Assume Z is your (n x r) embedding (e.g., TF-IDF + SVD)
Z <- scale(Z) # if not already standardized

# 1) Penalized silhouette across k:
ks <- seq(10, 500, by=10); sc <- data.frame()
for(k in ks){
  lab <- kmeans(Z, centers=min(k,nrow(Z)), nstart=10)$cluster
  ps  <- penalized_silhouette(Z, lab, dist_method="cosine",
                             min_cluster_size=2, lambda=0.3,
                             penalty="singleton", sample_n=2000)
  sc <- rbind(sc, data.frame(k=k, score=ps$score))
}
k_pen <- sc$k[which.max(sc$score)]

# 2) CH / DB / Gap:
# (use helpers on previous slide; pick k by max CH / min DB / Tibs rule for Gap)
```

Minimal Usage Examples - 2

```
# 3) Modularity across kNN:
knng <- seq(10, 300, by=10); Qtab <- data.frame()
for(kNN in knng){
  g <- build_knn_graph(Z, k=kNN, metric="cosine", mutual=TRUE, weights=TRUE)
  fit <- louvain_modularity(g)
  Qtab <- rbind(Qtab, data.frame(kNN=kNN, Q=fit$modularity))
}
kNN_best <- Qtab$kNN[which.max(Qtab$Q)]

# 4) Stability (average ARI):
stab_k <- sapply(ks, function(k) stability_via_resamples(Z, get_fit_kmeans, B=20, k=k))
k_stab <- ks[which.max(stab_k)]
```

Practical Guidance

- **Text ER:** cosine distances; prefer Penalized Silhouette ($m=2$, $\lambda=0.3$), Modularity (mutual kNN), Stability ($B=20$).
- **Scaling:** standardize features unless your embedding already is.
- **Speed:** sub-sample for silhouette & stability; increase B only when finalizing.
- **Blend:** use a weighted sum of two criteria when one is too flat (e.g., $Q + 0.5 \cdot \text{Stability}$).
- **Diagnostics:** track cluster size histograms to detect fragmentation.

Summary

- Silhouette can rise with k in high-dim data $\Rightarrow$ use **penalties**.
- CH/DB/Gap remain useful baselines with proper scaling.
- For graph methods, **modularity** and **stability** are strong selectors.
- Choose criterion(s) matched to data geometry and method family.